

Module 03: Feature Engineering I - The Cardinality Rules of Categorical Encoding

Cybernauts 2026 Datathon Campaign

May 2026

Abstract

Machine Learning models are mathematical engines; they only understand numbers. However, real-world data is messy and full of text categories (City, Job Title, Device Type). The way you transform these words into numbers determines whether your model finds a pattern or gets lost in a "Dimensionality Explosion." This note covers the three primary strategies for handling categories based on their cardinality.

Contents

1	Introduction to Cardinality	2
2	Strategy 1: One-Hot Encoding (OHE)	2
3	Strategy 2: Label and Ordinal Encoding	2
4	Strategy 3: Frequency / Count Encoding	2
5	Practice Problems	3

1 Introduction to Cardinality

Cardinality refers to the number of unique values in a categorical column.

- **Low Cardinality:** A few unique values (e.g., Gender, Subscription_Type).
- **High Cardinality:** Hundreds or thousands of unique values (e.g., City, User_ID, ZIP_Code).

The strategy you choose must match the cardinality of the feature.

2 Strategy 1: One-Hot Encoding (OHE)

One-Hot Encoding creates a new binary column (0 or 1) for every unique category in the feature.

When to use OHE

Only use this for **Low Cardinality** features (< 10 unique values). If you use OHE on a column with 500 cities, you will add 500 new columns to your dataset, making your data matrix massive and sparse (full of zeros), which ruins model performance.

```
1 # Using Pandas to One-Hot Encode
2 df = pd.get_dummies(df, columns=['Device_Type'], drop_first=True)
3
```

3 Strategy 2: Label and Ordinal Encoding

Label Encoding assigns a unique integer to each category (e.g., Apple → 0, Banana → 1). **Ordinal Encoding** is a special case where the integers follow a natural mathematical order.

When to use Ordinal

Use this when the categories have a **Ranked Order**. *Example:* Low → 0, Medium → 1, High → 2. This tells the model that "High" is mathematically "greater" than "Low."

```
1 # Manual Ordinal Mapping
2 mapping = {'Low': 0, 'Medium': 1, 'High': 2}
3 df['Priority'] = df['Priority'].map(mapping)
4
```

4 Strategy 3: Frequency / Count Encoding

This method replaces each category with either the total **count** or the **percentage** of times it appears in the dataset.

When to use Frequency

This is the gold standard for **High Cardinality** columns (like IP Addresses or Zip Codes). It captures the "importance" or "rarity" of a category without adding new columns.

```
1 # Frequency Encoding
2 freq = df['City'].value_counts(normalize=True)
3 df['City_Freq'] = df['City'].map(freq)
4
```

5 Practice Problems

P1: You have a column `Job_Title` with 1,200 unique titles. Should you use One-Hot Encoding? Why or why not?

Answer: No. Using OHE would create 1,200 new columns, leading to the "Curse of Dimensionality" and likely causing your model to overfit or crash due to memory limits.

P2: Which encoding method is best for a column `Education_Level` with values [High School, Bachelors, Masters, PhD]?

Answer: Ordinal Encoding. These categories have a clear logical progression where a PhD represents a higher level than High School.

P3: Why is Frequency Encoding useful for fraud detection?

Answer: In fraud detection, "rarity" is a signal. Frequency encoding tells the model exactly how rare an IP address or ZIP code is, helping it identify unusual (low-frequency) behavior.